

A very long title for my slides, which may span across two lines

Gianluca Baio

✉ g.baio@ucl.ac.uk

🌐 gianluca.statistica.it | 🌐 egon.stats.ucl.ac.uk/research/statistics-health-economics

🐙 github.com/giabaio | 🐙 github.com/StatisticsHealthEconomics

📧 @gianlubaio@mas.to | **in** [gianluca-baio](#)

20 July 2026

The name of the conference, Where it is

The name of the session

Disclaimer...



Home



Explore



Notifications



Messages



Lists



Bookmarks



Communities



Premium



Profile



More

Post

Manuela Joore @ManuelaJoore
Best opening sentence [#ISPOREurope](#) from Gianluca Baio:
“statisticians should rule the world and Bayesian statisticians should rule all statisticians”

Gianluca Baio @gianlubaio · Nov 4, 2019
Ready for our session on open source models & methods!

4:52 PM · Nov 4, 2019

2 2 16

Ed Wilson @EdCFWilson · Nov 4, 2019
And economists should rule the Bayesian statisticians 🙄

1 1 3

Gianluca Baio @gianlubaio · Nov 4, 2019
Now don't get ahead of yourself... 😂😂

2

Search

Relevant people

Manuela Joore @ManuelaJC Follows you Following
Professor Health Technology Assessment [#KEMTA](#)
[@maastrichtumc](#) [@maastrichtu](#)
[@CAPHRI_UM](#), [@healthinnovNL](#) & [@NICEcomms](#)

Gianluca Baio @gianlubaio
Dad to Kobi & Anna; Hubby to [@martablangiaro](#). Also, prof of Statistics & Health Economics [@ucl](#) & HoD [@stats_UCL](#). Some time to spare. mas.to/@gianlubaio

... Just so you know what you're about to get yourself into... 😊



Hello there 🖐️

This presentation will show you examples of what you can do with Quarto and **Reveal.js**, including:

- Presenting code and LaTeX equations
- Including computations in slide output
- Image, video, and iframe backgrounds
- Fancy transitions and animations
- Printing to PDF
 - Which is sometimes useful...

...and much more

Different output

- The default output is html via revealjs. This is managed by the option

```
format:  
ucl-revealjs
```

in the yml of the .qmd file.

- Alternatively, the slides can be produced in pdf directly via beamer. This is obtained by specifying

```
format:  
ucl-beamer
```

instead.

Here's a new slide without a title.

No logo

Here's a slide with no logo – just add `{ .no-logo }` to the title of the slide

Fun with colours & spaces

With a subtitle

This shows how to use colours in the slides. A bunch are pre-defined in the .css file and can be used simply as:

- The **predictive** distribution can be computed as follows.

More can be created by simply modifying the .css file to add more HEX codes.

- For example use

```
.uclblue  
color: #002855;
```

(though this colour is already defined)

Can add vertical space using fragments

```
::: style="margin-top: 40px"  
Can add vertical space using `fragments`  
:::
```

Some maths

We can also include maths, using standard LaTeX:

$$p(y \mid \mathcal{D}) = \int p(y \mid \boldsymbol{\theta})p(\boldsymbol{\theta} \mid \mathcal{D})d\boldsymbol{\theta}$$

To use colours with LaTeX, needs a little hack: `\class{color}{expression}`

You can define a probabilistic assumption as

$$y \mid \boldsymbol{\theta} \sim \text{Poisson}(\boldsymbol{\theta})$$

but in line it would look like this $y \sim \text{Normal}(\mu, \sigma^2)$

These use the predefined LaTeX macros, saved in the file `latex_macros.html`. More can be (re)defined by simply adding to the file, which gets pre-loaded at the beginning of the resulting `.html` output; example: `\DIC=2345` is typeset as $\text{DIC} = 2345$.

For instance, the command `\dnorm` translates to the text Normal (typeset in the current font). But can also use `\txt{Some text}`, which gets rendered as Some text — it's like `\text{...}` but with the current font

Pretty Code

(Slide 1)

- Over 20 syntax highlighting themes available
- Default theme optimized for accessibility

```
# Define a server for the Shiny app
function(input, output)

# Fill in the spot we created for a plot
output$phonePlot <- renderPlot(
# Render a barplot
)
```

Learn more: [Syntax Highlighting](#)

Code Animations

(Slide 2)

- Over 20 syntax highlighting themes available
- Default theme optimized for accessibility

```
# Define a server for the Shiny app
function(input, output)

# Fill in the spot we created for a plot
output$phonePlot <- renderPlot(
# Render a barplot
barplot(WorldPhones[,input$region]*1000,
main=input$region,
ylab="Number of Telephones",
xlab="Year")
)
```

Learn more: [Code Animations](#)

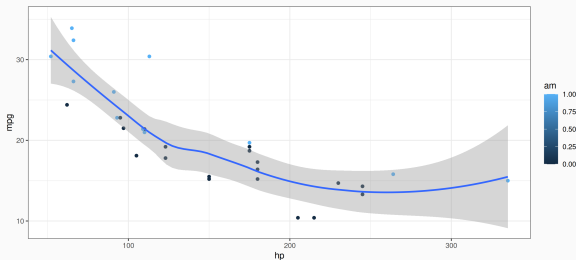
Text formatting

Blockquote

More quoted text

Executable Code

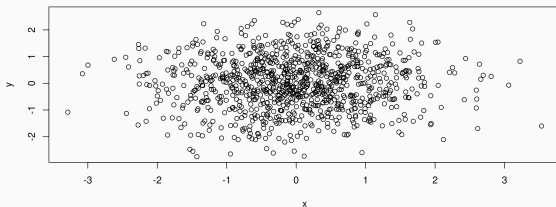
```
library(ggplot2)
ggplot(mtcars, aes(hp, mpg, color = am)) +
  geom_point() + geom_smooth(formula = y ~ x, method = "loess")
```



Learn more: [Executable Code](#)

A base plot

```
plot(rnorm(1000),rnorm(1000),xlab="x",ylab="y")
```



A section

A section

Using the tag `{.divider}` makes for some nicer css formatting (especially under `ucl-revealjs`)

Column Layout

Arrange content into columns of varying widths:

Motor Trend Car Road Tests

The data was extracted from the 1974 Motor Trend US magazine, and comprises fuel consumption and 10 aspects of automobile design and performance for 32 automobiles.

	mpg	cyl	disp	hp	wt
Mazda RX4	21.0	6	160	110	2.620
Mazda RX4 Wag	21.0	6	160	110	2.875
Datsun 710	22.8	4	108	93	2.320
Hornet 4 Drive	21.4	6	258	110	3.215
Hornet Sportabout	18.7	8	360	175	3.440
Valiant	18.1	6	225	105	3.460

Learn more: [Multiple Columns](#)

Incremental Lists

Lists can optionally be displayed incrementally:

1. First item

Incremental Lists

Lists can optionally be displayed incrementally:

1. First item
2. Second item

Incremental Lists

Lists can optionally be displayed incrementally:

1. First item
2. Second item
3. Third item

Incremental Lists

Lists can optionally be displayed incrementally:

1. First item
2. Second item
3. Third item

Incremental Lists

Lists can optionally be displayed incrementally:

1. First item
2. Second item
3. Third item

Insert pauses to make other types of content display incrementally.

Learn more: [Incremental Lists](#)

Fragments

Incremental text display and animation with fragments:

Extra fragment styling / nesting / ordering options

Fade in

Fragments

Incremental text display and animation with fragments:

Extra fragment styling / nesting / ordering options

Fade in

Slide up while fading in

Fragments

Incremental text display and animation with fragments:

Extra fragment styling / nesting / ordering options

Fade in

Slide up while fading in

Slide left while fading in

Fragments

Incremental text display and animation with fragments:

Extra fragment styling / nesting / ordering options

Fade in

Slide up while fading in

Slide left while fading in

Fade in then semi out

Fragments

Incremental text display and animation with fragments:

Extra fragment styling / nesting / ordering options

Fade in

Slide up while fading in

Slide left while fading in

Fade in then semi out

Fragments

Incremental text display and animation with fragments:

Extra fragment styling / nesting / ordering options

Fade in

Slide up while fading in

Slide left while fading in

Fade in then semi out

Strike

Highlight red (but green and blue are also built-in)

Fragments

Incremental text display and animation with fragments:

Extra fragment styling / nesting / ordering options

Fade in

Slide up while fading in

Slide left while fading in

Fade in then semi out

Strike

Highlight red (but green and blue are also built-in)

Fragments

Incremental text display and animation with fragments:

Extra fragment styling / nesting / ordering options

Fade in

Slide up while fading in

Slide left while fading in

Fade in then semi out

Strike

Highlight red (but green and blue are also built-in)

Fragments

Incremental text display and animation with fragments:

Extra fragment styling / nesting / ordering options

Fade in

Slide up while fading in

Slide left while fading in

Fade in then semi out

Strike

Highlight red (but green and blue are also built-in)

Highlight custom colour (defined in scss file or can be added on the fly)

```
```css
#| echo: false
.reveal .slides section .fragment.highlight-itarred
opacity: 1;
visibility: inherit;
```

# Fragments (in beamer)

- The file `beamer-meta.lua` defines a filter to map the code `::: {.fragment .highlight-XXX}`, which is specific to `revealjs`
- More colours can be defined and added to the filter, in the part

```
local highlight_colors = {
 ["highlight-red"] = "red",
 ["highlight-green"] = "green",
 ["highlight-blue"] = "blue",
 ["highlight-itarred"] = "italian-red",
 ["highlight-itagreen"] = "italian-green",
 ["new-colour"] = "new-colour"
}
```

and the `new-colour` must also be defined in the `ucl-beamer.tex` template (both under `_extensions/ucl/beamer`)